ADVANCED NUMERIC PATTERN GENERATOR

Python Activity-Based Learning (ABL) Assignment — Sample Implementation

1. Problem Statement

Create a Python program that generates either a hollow numeric diamond or a symmetric numeric pattern based on user-specified dimensions. The program must validate invalid dimensions and align the output carefully.

2. Objectives

• Generate a hollow numeric diamond with a centered outline.

• Generate a symmetric numeric diamond pattern.

• Validate the menu choice and require a positive whole-number dimension.

• Handle invalid text and non-positive dimensions without crashing.

• Display neatly aligned output in the terminal.

3. Assumptions

The dimension is a positive integer. Pattern option 1 produces the hollow diamond; option 2 produces a symmetric numeric diamond. A size of 5 is used for the sample output. Very large sizes are allowed but may produce wide output.

4. Algorithm

1. Display the pattern menu.

2. Read the user's choice and validate that it is 1 or 2.

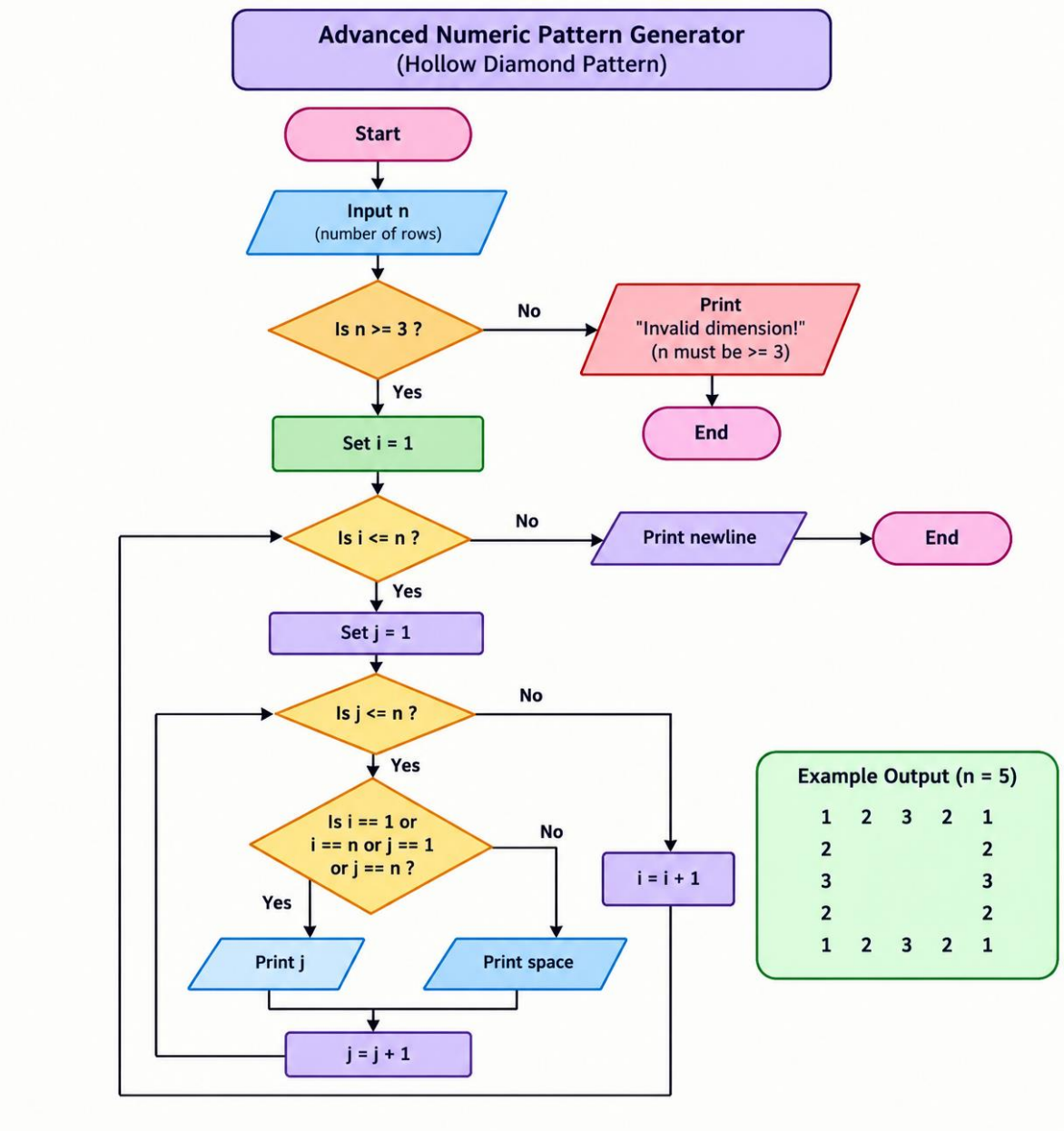
3. Read the dimension and validate that it is a positive integer.

4. For a hollow diamond, print increasing rows followed by decreasing rows, using outer and inner spaces.

5. For a symmetric pattern, print increasing number sequences and then mirror them in reverse order.

6. Display the pattern and stop.

5. Flowchart



6. Pseudocode

BEGIN

PRINT "Advanced Numeric Pattern Generator"

PRINT "Enter the dimension (n): "

INPUT n

```
  IF n < 3 THEN
    PRINT "Invalid dimension!"
    PRINT "Dimension must be at least 3"
  ELSE

    FOR i = 1 TO n

      FOR j = 1 TO n

        IF i == 1 OR i == n OR j == 1 OR j == n THEN
          PRINT j WITHOUT NEWLINE
        ELSE
          PRINT SPACE WITHOUT NEWLINE
        END IF

      END FOR

      PRINT NEWLINE

    END FOR

  END IF

END
```

7. Python Source Code

```
def hollow_diamond(n):
    if n < 3:
        print("Invalid dimension! Dimension must be at least 3.")
        return

    # Upper half of the diamond
    for i in range(1, n + 1):
        for j in range(n - i):
            print(" ", end="")

        for j in range(1, 2 * i):
            if j == 1 or j == 2 * i - 1:
                print(j if j <= i else 2 * i - j, end="")
            else:
                print(" ", end="")

        print()

    # Lower half of the diamond
    for i in range(n - 1, 0, -1):
        for j in range(n - i):
            print(" ", end="")

        for j in range(1, 2 * i):
            if j == 1 or j == 2 * i - 1:
```

```
            print(j if j <= i else 2 * i - j, end="")
        else:
            print(" ", end="")

    print()
```

```
try:
    n = int(input("Enter the dimension (minimum 3): "))
    hollow_diamond(n)
except ValueError:
    print("Invalid input! Please enter an integer.")
```

8. Sample Test and Output

```
Enter the dimension (minimum 3): 7
      1
     1 1
    1   1
   1     1
  1       1
 1         1
1           1
 1         1
  1       1
   1     1
    1   1
     1 1
      1
```

9. Validation Test Cases Input

1. **Test Case 1 – Valid Input:** Enter dimension 5. Expected output: A hollow diamond pattern should be generated with proper spacing and alignment.

2. **Test Case 2 – Minimum Valid Dimension:** Enter dimension 3. Expected output: A valid hollow diamond pattern should be displayed.

3. **Test Case 3 – Invalid Dimension:** Enter dimension 2. Expected output: Display “Invalid dimension! Dimension must be at least 3.”

4. **Test Case 4 – Zero Input:** Enter dimension `0`. Expected output: Display an invalid dimension message.

5. **Test Case 5 – Negative Input:** Enter dimension `-1`. Expected output: Display an invalid dimension message.

6. **Test Case 6 – Non-numeric Input:** Enter `abc`. Expected output: Display “Invalid input! Please enter an integer.”

7. **Test Case 7 – Decimal Input:** Enter `3.5`. Expected output: Display an invalid input message.

8. **Test Case 8 – Larger Dimension:** Enter dimension `10`. Expected output: Generate a larger hollow diamond pattern with proper alignment.

9. **Test Case 9 – Boundary Value:** Enter dimension 4. Expected output: Generate a valid symmetric hollow diamond pattern.

10. **Test Case 10 – Empty Input:** Do not enter any value. Expected output: Display an input validation error instead of terminating unexpectedly.

10. Conclusion

The program demonstrates nested loops, string formatting, input validation, and pattern generation. It supports two pattern types and safely rejects invalid choices and invalid dimensions.